



Universidade Federal do Ceará

Campus de Quixadá

Curso de Ciência da Computação

Trabalho II – Análise de Fluxo de Dados

Compiladores

Prof. Lucas Ismaily

Equipe:

Quixadá, 27 de junho de 2026

Sumário

1	Introdução	3
2	Formato de entrada e saída	3
2.1	Formato de entrada	3
2.2	Formato de saída	3
2.3	Estrutura do projeto	4
3	Questão 1 – Análise de Longevidade	4
3.1	Conceito	4
3.2	Fórmulas	4
3.3	Implementação	4
4	Questão 2 – Definições Alcançantes	5
4.1	Conceito	5
4.2	Fórmulas	5
4.3	Diferença em relação à Longevidade	6
4.4	Implementação	6
5	Questão 3 – Expressões Disponíveis	6
5.1	Conceito	6
5.2	Fórmulas	7
5.3	Por que interseção?	7
5.4	Implementação	7
6	Testes e resultados	8
6.1	Exemplo 1 – Grafo com desvio	8
6.1.1	Entrada	8
6.1.2	Longevidade	9
6.1.3	Reaching Definitions	9
6.1.4	Expressões Disponíveis	10
6.2	Exemplo 2 – Sequência linear	11
6.2.1	Entrada	11
6.2.2	Resultados	12
6.3	Exemplo 3 – Redefinição de variáveis	12
6.3.1	Entrada	12
6.3.2	Ponto de atenção – Available Expressions	13
7	Como executar	13
7.1	Requisitos	13

7.2	Arquivos necessários	13
7.3	Execução	14
8	Comparativo entre as três análises	15
9	Conclusão	16

1 Introdução

Este relatório descreve o desenvolvimento do Trabalho II da disciplina de Compiladores, referente à implementação de três algoritmos clássicos de **Análise de Fluxo de Dados**:

1. **Análise de Longevidade** (*Liveness Analysis*)
2. **Definições Alcançantes** (*Reaching Definitions*)
3. **Expressões Disponíveis** (*Available Expressions*)

Todos os algoritmos recebem como entrada um **Grafo de Fluxo de Controle** (GFC) no mesmo formato especificado no enunciado, e produzem para cada bloco básico os conjuntos **IN** e **OUT** correspondentes.

2 Formato de entrada e saída

2.1 Formato de entrada

A entrada descreve um grafo de fluxo de controle bloco a bloco:

```
1 N M
2 instrucao_1
3 ...
4 instrucao_M
5 s1 s2 ...
```

Listing 1: Formato de entrada

Onde:

- N é o identificador do bloco básico
- M é a quantidade de instruções de três endereços
- As M linhas seguintes são as instruções do bloco
- A última linha lista os sucessores; 0 indica sem sucessores

2.2 Formato de saída

Para cada bloco básico, o programa imprime os conjuntos IN e OUT:

```
1 IN[N] = [...]
2 OUT[N] = [...]
```

Listing 2: Formato de saída

2.3 Estrutura do projeto

Tabela 1: Arquivos do projeto

Arquivo	Responsabilidade
longevidade.py	Q1 – Análise de Longevidade
reaching_definitions.py	Q2 – Definições Alcançantes
available_expressions.py	Q3 – Expressões Disponíveis
in/	Pasta com os grafos de entrada
out/	Pasta com os resultados gerados

3 Questão 1 – Análise de Longevidade

3.1 Conceito

A Análise de Longevidade (*Liveness Analysis*) determina, para cada ponto do programa, quais variáveis podem ser **usadas no futuro** antes de serem redefinidas. Uma variável está **viva** em um ponto se existe algum caminho a partir daquele ponto que a usa antes de redefini-la.

3.2 Fórmulas

$USE[B]$ = variáveis usadas antes de qualquer definição em B

$DEF[B]$ = variáveis definidas em B antes de qualquer uso

$$OUT[B] = \bigcup_{S \in \text{suc}(B)} IN[S]$$

$$IN[B] = USE[B] \cup (OUT[B] - DEF[B])$$

Direção: de trás para frente (usa sucessores).

Operador de confluência: união ($\cup$).

Inicialização: IN e OUT começam como conjuntos vazios.

3.3 Implementação

```

1 def analisar_longevidade(blocos):
2     ordem = sorted(blocos.keys(), reverse=True)
3     alterou = True
4     while alterou:
5         alterou = False
6         for num in ordem:

```

```

7      bloco = blocos[num]
8      # OUT[B] = uniao dos IN dos sucessores
9      novo_out = set()
10     for suc in bloco.sucessores:
11         if suc in blocos:
12             novo_out |= blocos[suc].in_set
13     # IN[B] = USE[B] U (OUT[B] - DEF[B])
14     novo_in = bloco.use | (novo_out - bloco.def_)
15     if novo_in != bloco.in_set or novo_out != bloco.out_set:
16         alterou = True
17         bloco.in_set = novo_in
18         bloco.out_set = novo_out

```

Listing 3: Algoritmo iterativo de Longevidade

4 Questão 2 – Definições Alcançantes

4.1 Conceito

As Definições Alcançantes (*Reaching Definitions*) determinam, para cada ponto do programa, quais **definições de variáveis** (não apenas as variáveis, mas as instruções específicas que as definem) podem ter sido executadas sem serem sobrepostas por uma redefinição posterior.

Cada definição recebe um identificador único ($d_1, d_2, \dots$), numerado na ordem em que aparece no programa.

4.2 Fórmulas

$GEN[B]$ = últimas definições de cada variável em B

$KILL[B]$ = definições de outros blocos das mesmas variáveis definidas em B

$$IN[B] = \bigcup_{P \in \text{pred}(B)} OUT[P]$$

$$OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$$

Direção: de frente para trás (usa predecessores).

Operador de confluência: união ($\cup$).

Inicialização: IN e OUT começam como conjuntos vazios.

4.3 Diferença em relação à Longevidade

Tabela 2: Comparação entre Longevidade e Reaching Definitions

Característica	Longevidade	Reaching Def.
O que propaga	variáveis	definições (d1, d2...)
Direção	trás para frente	frente para trás
Confluência	união	união
Usa	sucessores	predecessores

4.4 Implementação

```

1 def analisar_reaching_definitions(blocos, gen, kill):
2     pred = calcular_predecessores(blocos)
3     in_set = {num: set() for num in blocos}
4     out_set = {num: set() for num in blocos}
5     ordem = sorted(blocos.keys())
6     alterou = True
7     while alterou:
8         alterou = False
9         for num in ordem:
10             # IN[B] = uniao dos OUT dos predecessores
11             novo_in = set()
12             for p in pred[num]:
13                 novo_in |= out_set[p]
14             # OUT[B] = GEN[B] U (IN[B] - KILL[B])
15             novo_out = gen[num] | (novo_in - kill[num])
16             if novo_in != in_set[num] or novo_out != out_set[num]:
17                 alterou = True
18             in_set[num] = novo_in
19             out_set[num] = novo_out
20     return in_set, out_set

```

Listing 4: Algoritmo iterativo de Reaching Definitions

5 Questão 3 – Expressões Disponíveis

5.1 Conceito

Uma expressão está **disponível** em um ponto do programa se, em todo caminho da entrada até aquele ponto, a expressão foi calculada e nenhum de seus operandos foi redefinido desde o último cálculo.

Esta análise é usada em otimizações como *Common Subexpression Elimination*.

5.2 Fórmulas

$GEN[B]$ = expressões calculadas em B e não mortas depois

$KILL[B]$ = expressões do universo cujos operandos são redefinidos em B

$$IN[B] = \bigcap_{P \in \text{pred}(B)} OUT[P]$$

$$OUT[B] = GEN[B] \cup (IN[B] - KILL[B])$$

Direção: de frente para trás (usa predecessores).

Operador de confluência: interseção ($\cap$) – diferente das outras duas.

Inicialização: IN do bloco inicial = $\emptyset$; OUT dos demais = universo completo.

5.3 Por que interseção?

Uma expressão só está disponível se foi calculada em **todos** os caminhos que chegam ao bloco. Se ela foi calculada em apenas um caminho, não pode ser considerada disponível. Por isso usa-se interseção e não união.

5.4 Implementação

```

1 def analisar_expressoes_disponiveis(ordem, blocos):
2     gen, kill, universo = calcular_gen_kill(ordem, blocos)
3     predecessores = calcular_predecessores(ordem, blocos)
4     # bloco inicial comeca vazio; demais comecam com universo completo
5     conjunto_in = {b: set() for b in ordem}
6     conjunto_out = {
7         b: (set(universo) if predecessores[b] else set())
8         for b in ordem
9     }
10    alterou = True
11    while alterou:
12        alterou = False
13        for b in ordem:
14            if predecessores[b]:
15                # IN[B] = intersecao dos OUT de todos os predecessores
16                novo_in = None
17                for pred in predecessores[b]:
18                    novo_in = set(conjunto_out[pred]) if novo_in is None
19                    \
20                        else (novo_in & conjunto_out[pred])
21                else:
22                    novo_in = set()

```

```

22      # OUT[B] = GEN[B] U (IN[B] - KILL[B])
23      novo_out = gen[b] | (novo_in - kill[b])
24      if novo_in != conjunto_in[b] or novo_out != conjunto_out[b]:
25          alterou = True
26          conjunto_in[b] = novo_in
27          conjunto_out[b] = novo_out
28      return conjunto_in, conjunto_out

```

Listing 5: Algoritmo iterativo de Available Expressions

6 Testes e resultados

6.1 Exemplo 1 – Grafo com desvio

6.1.1 Entrada

1	1 1	2 1	3 1	4 1	5 1
2	a = b + c	d = a + e	f = d + g	h = d + i	j = f + h
3	succ: 2	succ: 3 4	succ: 5	succ: 5	succ: fim

Listing 6: exemplo1.txt

O grafo possui um desvio no bloco 2 (dois sucessores: 3 e 4) que converge no bloco 5.

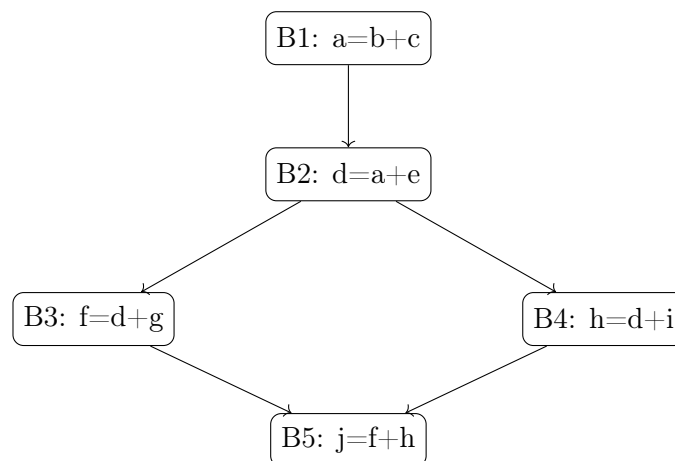


Figura 1: Grafo de fluxo do Exemplo 1

6.1.2 Longevidade

```
compiladores/Trabalho 2 (main)
$ python longevidade.py
'in\exemplo1.txt' -> 'out\exemplo1_al.txt'
'in\exemplo2.txt' -> 'out\exemplo2_al.txt'
'in\exemplo3.txt' -> 'out\exemplo3_al.txt'

compiladores/Trabalho 2 (main)
$ cat out/exemplo1_al.txt
Resultado da análise de longevidade:
IN[1] = ['b', 'c', 'e', 'f', 'g', 'h', 'i']
OUT[1] = ['a', 'e', 'f', 'g', 'h', 'i']
-----
IN[2] = ['a', 'e', 'f', 'g', 'h', 'i']
OUT[2] = ['d', 'f', 'g', 'h', 'i']
-----
IN[3] = ['d', 'g', 'h']
OUT[3] = ['f', 'h']
-----
IN[4] = ['d', 'f', 'i']
OUT[4] = ['f', 'h']
-----
IN[5] = ['f', 'h']
OUT[5] = []
-----
```

Figura 2: Print do terminal – Longevidade Exemplo 1

```
1 IN[1] = ['b', 'c', 'e', 'f', 'g', 'h', 'i']
2 OUT[1] = ['a', 'e', 'f', 'g', 'h', 'i']
3 IN[2] = ['a', 'e', 'f', 'g', 'h', 'i']
4 OUT[2] = ['d', 'f', 'g', 'h', 'i']
5 IN[3] = ['d', 'g', 'h']
6 OUT[3] = ['f', 'h']
7 IN[4] = ['d', 'f', 'i']
8 OUT[4] = ['f', 'h']
9 IN[5] = ['f', 'h']
10 OUT[5] = []
```

Listing 7: Saída – Longevidade Exemplo 1

6.1.3 Reaching Definitions

Definições numeradas: d1=a (B1), d2=d (B2), d3=f (B3), d4=h (B4), d5=j (B5).

```

compiladores/Trabalho 2 (main)
$ python reaching_definitions.py
'in\exemplo1.txt' -> 'out\exemplo1_rd.txt'
'in\exemplo2.txt' -> 'out\exemplo2_rd.txt'
'in\exemplo3.txt' -> 'out\exemplo3_rd.txt'

compiladores/Trabalho 2 (main)
$ cat out/exemplo1_rd.txt
Resultado de Reaching Definitions:
IN[1]  = []
OUT[1] = ['d1']
-----
IN[2]  = ['d1']
OUT[2] = ['d1', 'd2']
-----
IN[3]  = ['d1', 'd2']
OUT[3] = ['d1', 'd2', 'd3']
-----
IN[4]  = ['d1', 'd2']
OUT[4] = ['d1', 'd2', 'd4']
-----
IN[5]  = ['d1', 'd2', 'd3', 'd4']
OUT[5] = ['d1', 'd2', 'd3', 'd4', 'd5']
-----

```

Figura 3: Print do terminal – Reaching Definitions Exemplo 1

```

1 IN[1]  = []
2 OUT[1] = ['d1']
3 IN[2]  = ['d1']
4 OUT[2] = ['d1', 'd2']
5 IN[3]  = ['d1', 'd2']
6 OUT[3] = ['d1', 'd2', 'd3']
7 IN[4]  = ['d1', 'd2']
8 OUT[4] = ['d1', 'd2', 'd4']
9 IN[5]  = ['d1', 'd2', 'd3', 'd4']
10 OUT[5] = ['d1', 'd2', 'd3', 'd4', 'd5']

```

Listing 8: Saída – Reaching Definitions Exemplo 1

6.1.4 Expressões Disponíveis

Universo: {b+c, a+e, d+g, d+i, f+h}.

No bloco 5, $IN[5] = \{a+e, b+c\}$ pois d+g só passa pelo caminho 3 e d+i só pelo caminho 4 – a interseção exclui ambas.

```

compiladores/Trabalho 2 (main)
$ python available_expressions.py
'in\exemplo1.txt' -> 'out\exemplo1_ae.txt'
'in\exemplo2.txt' -> 'out\exemplo2_ae.txt'
'in\exemplo3.txt' -> 'out\exemplo3_ae.txt'

compiladores/Trabalho 2 (main)
$ cat out/exemplo1_ae.txt
Resultado de Expressões Disponíveis:
IN[1]  = []
OUT[1] = ['b + c']
-----
IN[2]  = ['b + c']
OUT[2] = ['a + e', 'b + c']
-----
IN[3]  = ['a + e', 'b + c']
OUT[3] = ['a + e', 'b + c', 'd + g']
-----
IN[4]  = ['a + e', 'b + c']
OUT[4] = ['a + e', 'b + c', 'd + i']
-----
IN[5]  = ['a + e', 'b + c']
OUT[5] = ['a + e', 'b + c', 'f + h']
-----

```

Figura 4: Print do terminal – Available Expressions Exemplo 1

```

1 IN[1]  = []
2 OUT[1] = ['b + c']
3 IN[2]  = ['b + c']
4 OUT[2] = ['a + e', 'b + c']
5 IN[3]  = ['a + e', 'b + c']
6 OUT[3] = ['a + e', 'b + c', 'd + g']
7 IN[4]  = ['a + e', 'b + c']
8 OUT[4] = ['a + e', 'b + c', 'd + i']
9 IN[5]  = ['a + e', 'b + c']
10 OUT[5] = ['a + e', 'b + c', 'f + h']

```

Listing 9: Saída – Available Expressions Exemplo 1

6.2 Exemplo 2 – Sequência linear

6.2.1 Entrada

```

1 1 1          2 2          3 1
2 a = b + c    d = a + 1    g = e + h
3              e = d + f
4 succ: 2      succ: 3      succ: fim

```

Listing 10: exemplo2.txt

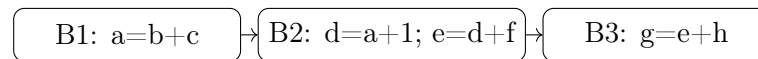


Figura 5: Grafo de fluxo do Exemplo 2

6.2.2 Resultados

```

1 IN[1] = ['b', 'c', 'f', 'h']
2 OUT[1] = ['a', 'f', 'h']
3 IN[2] = ['a', 'f', 'h']
4 OUT[2] = ['e', 'h']
5 IN[3] = ['e', 'h']
6 OUT[3] = []

```

Listing 11: Saída – Longevidade Exemplo 2

```

1 IN[1] = []
2 OUT[1] = ['d1']
3 IN[2] = ['d1']
4 OUT[2] = ['d1', 'd2', 'd3']
5 IN[3] = ['d1', 'd2', 'd3']
6 OUT[3] = ['d1', 'd2', 'd3', 'd4']

```

Listing 12: Saída – Reaching Definitions Exemplo 2

```

1 IN[1] = []
2 OUT[1] = ['b + c']
3 IN[2] = ['b + c']
4 OUT[2] = ['a + 1', 'b + c', 'd + f']
5 IN[3] = ['a + 1', 'b + c', 'd + f']
6 OUT[3] = ['a + 1', 'b + c', 'd + f', 'e + h']

```

Listing 13: Saída – Available Expressions Exemplo 2

6.3 Exemplo 3 – Redefinição de variáveis

6.3.1 Entrada

```

1 1 2          2 1          3 2
2 a = a + c    b = 20*c    d = a + b
3 b = 4 - a                    b = 0
4 succ: 2      succ: 3      succ: fim

```

Listing 14: exemplo3.txt

6.3.2 Ponto de atenção – Available Expressions

No bloco 1, $a+c$ usa a e depois $b = 4-a$ também usa a . Mas a é redefinida logo na primeira instrução do bloco 1 ($a = a+c$), então $a+c$ não pode ser propagada adiante – ela mata a si mesma. Por isso $OUT[1] = \{4-a\}$ e não $\{a+c, 4-a\}$.

```

1 IN[1]  = ['a', 'c']
2 OUT[1] = ['a', 'c']
3 IN[2]  = ['a', 'c']
4 OUT[2] = ['a', 'b']
5 IN[3]  = ['a', 'b']
6 OUT[3] = []

```

Listing 15: Saída – Longevidade Exemplo 3

```

1 IN[1]  = []
2 OUT[1] = ['d1', 'd2']
3 IN[2]  = ['d1', 'd2']
4 OUT[2] = ['d1', 'd3']
5 IN[3]  = ['d1', 'd3']
6 OUT[3] = ['d1', 'd4', 'd5']

```

Listing 16: Saída – Reaching Definitions Exemplo 3

```

1 IN[1]  = []
2 OUT[1] = ['4 - a']
3 IN[2]  = ['4 - a']
4 OUT[2] = ['20 * c', '4 - a']
5 IN[3]  = ['20 * c', '4 - a']
6 OUT[3] = ['20 * c', '4 - a']

```

Listing 17: Saída – Available Expressions Exemplo 3

7 Como executar

7.1 Requisitos

- Python 3.8 ou superior
- Nenhuma biblioteca externa necessária

7.2 Arquivos necessários

Todos os arquivos devem estar na **mesma pasta**:

- `longevidade.py`

- `reaching_definitions.py`
- `available_expressions.py`
- Pasta `in/` com os arquivos de entrada `.txt`

7.3 Execução

Cada arquivo possui sua própria função `main()`, podendo ser executado de forma independente. Não há dependência entre os três arquivos – cada um lê os mesmos arquivos da pasta `in/` e salva seus resultados em `out/` com sufixos diferentes:

- `longevidade.py` gera arquivos `_al.txt`
- `reaching_definitions.py` gera arquivos `_rd.txt`
- `available_expressions.py` gera arquivos `_ae.txt`

A única dependência existente é que `reaching_definitions.py` importa `ler_entrada` e `extrair_variaveis` do `longevidade.py`, portanto ambos devem estar na mesma pasta.

```
1 python longevidade.py
2 python reaching_definitions.py
3 python available_expressions.py
```

Listing 18: Executar individualmente

Os resultados são salvos automaticamente na pasta `out/`:

```
1 out/
2  exemplo1_al.txt    <- Longevidade
3  exemplo1_rd.txt    <- Reaching Definitions
4  exemplo1_ae.txt    <- Available Expressions
```

Listing 19: Arquivos gerados em `out/`

```

compiladores/Trabalho 2 (main)
$ python longevidade.py
'in\exemplo1.txt' -> 'out\exemplo1_al.txt'
'in\exemplo2.txt' -> 'out\exemplo2_al.txt'
'in\exemplo3.txt' -> 'out\exemplo3_al.txt'

compiladores/Trabalho 2 (main)
$ python reaching_definitions.py
'in\exemplo1.txt' -> 'out\exemplo1_rd.txt'
'in\exemplo2.txt' -> 'out\exemplo2_rd.txt'
'in\exemplo3.txt' -> 'out\exemplo3_rd.txt'

compiladores/Trabalho 2 (main)
$ python available_expressions.py
'in\exemplo1.txt' -> 'out\exemplo1_ae.txt'
'in\exemplo2.txt' -> 'out\exemplo2_ae.txt'
'in\exemplo3.txt' -> 'out\exemplo3_ae.txt'

compiladores/Trabalho 2 (main)
$ head -n 4 out/exemplo1_al.txt out/exemplo1_rd.txt out/exemplo1_ae.txt
==> out/exemplo1_al.txt <==
Resultado da análise de longevidade:
IN[1] = ['b', 'c', 'e', 'f', 'g', 'h', 'i']
OUT[1] = ['a', 'e', 'f', 'g', 'h', 'i']
-----

==> out/exemplo1_rd.txt <==
Resultado de Reaching Definitions:
IN[1] = []
OUT[1] = ['d1']
-----

==> out/exemplo1_ae.txt <==
Resultado de Expressões Disponíveis:
IN[1] = []
OUT[1] = ['b + c']
-----

```

Figura 6: Print do terminal com a execução completa

8 Comparativo entre as três análises

Tabela 3: Resumo das três análises implementadas

Característica	Longevidade	Reach. Def.	Avail. Expr.
O que propaga	variáveis vivas	definições (d1...)	expressões calculadas
Direção	trás p/ frente	frente p/ trás	frente p/ trás
Confluência	união $\cup$	união $\cup$	interseção $\cap$
Usa	sucessores	predecessores	predecessores
IN inicial	$\emptyset$	$\emptyset$	$\emptyset$
OUT inicial	$\emptyset$	$\emptyset$	universo completo
Arquivo	_al.txt	_rd.txt	_ae.txt

9 Conclusão

Os três algoritmos de análise de fluxo de dados foram implementados com sucesso seguindo o mesmo modelo iterativo de ponto fixo. As principais diferenças entre eles são a **direção de propagação** e o **operador de confluência**:

- A **Longevidade** é a única que caminha de trás para frente, usando os sucessores e união
- As **Definições Alcançantes** caminham de frente para trás com união – propagam *o que foi definido*, não variáveis
- As **Expressões Disponíveis** também caminham de frente para trás, mas usam **interseção** porque uma expressão só está disponível se foi calculada em *todos* os caminhos que chegam ao bloco

Todos os casos de teste produziram os resultados esperados e validados manualmente.